

幻尔科技

机械臂基础控制手册



Hiwonder 官方网站



版本号	修改日期	修改摘要
V1.0	2025/07/24	初次发布

目录

1. 舵机控制节点	1
1.1 舵机控制节点介绍	2
1.2 通过话题控制舵机	3
1.3 开启 APP 自启动服务	3
1.4 程序分析	5
2. LED 控制节点	7
2.1 LED 控制节点介绍	7
2.2 通过话题控制 LED	8
2.3 开启 APP 自启动服务	10
2.4 程序分析	10
3. 蜂鸣器控制节点	12
3.1 蜂鸣器控制节点介绍	13
3.2 通过话题控制蜂鸣器	14
3.3 开启 APP 自启动服务	15
3.4 程序分析	15
4. 上位机控制	18
4.1 上位机介绍	18
4.2 动作组编辑及调用	23
4.3 串联动作组	28
4.4 偏差调节	31
5. 舵机调试工具的使用	41
5.1 舵机调试界面简介	41
5.2 舵机调试操作步骤	43

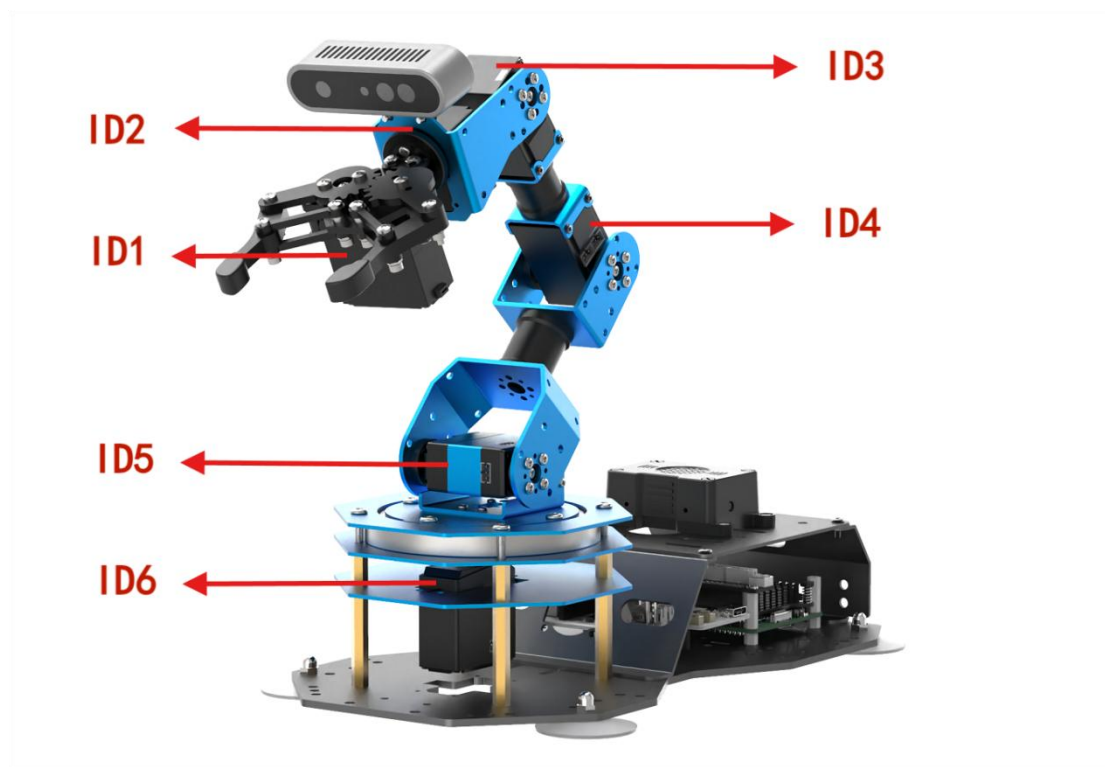
机械臂基础控制手册

机械臂是机械技术领域中得到最广泛实际应用的自动化机械装置，在工业制造、医学治疗、娱乐服务、军事、半导体制造以及太空探索等领域都能见到它的身影。

尽管它们的形态各有不同，但它们都有一个共同的特点，就是能够接受指令，精确地定位到三维（或二维）空间上的某一点进行作业，搭载了视觉系统的机械臂在此基础上功能将更加完善，本文档内容将以 ArmPi Ultra 为例，讲解本款机械臂的基础结构以及动作操控，其他版本的机械臂控制方式也是一样的。

1. 舵机控制节点

机械臂上一共有六个舵机，我们通过树莓派主控板，STM32 单片机进行通信，通过串口上对舵机进行控制。为了能够更加方便和规范的控制舵机，我们就需要设置一个 ROS 节点作为各个舵机控制的接口。



1.1 舵机控制节点介绍

1.1.1 准备工作

- 1) 根据“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意事项”内容安装好机械臂平台。
- 2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。
- 3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

1.1.2 启动舵机控制节点

- 1) 单击，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

- 2) 输入命令，启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

- 3) 打开一个新的命令栏，输入命令，查看机械臂节点启动后的话题，bus_servo/set_position 话题用于控制舵机运动。

```
ros2 topic list
```

```
> ros2 topic list
/controller_manager/servo_states
/depth_cam/rgb/camera_info
/depth_cam/rgb/image_raw
/parameter_events
/ros_robot_controller/battery
/ros_robot_controller/bus_servo/set_position
/ros_robot_controller/bus_servo/set_state
/ros_robot_controller/button
/ros_robot_controller/enable_reception
/ros_robot_controller/imu_raw
/ros_robot_controller/joy
/ros_robot_controller/pwm_servo/set_state
/ros_robot_controller/sbus
/ros_robot_controller/set_buzzer
/ros_robot_controller/set_conveyor_speed
/ros_robot_controller/set_led
/ros_robot_controller/set_motor_speeds
/ros_robot_controller/set_oled
/ros_robot_controller/set_stepper_subdivision
/rosout
/servo_controller
```

1.2 通过话题控制舵机

1.2.1 在终端控制

打开一个新的命令栏，输入命令

```
ros2 topic pub /ros_robot_controller/bus_servo/set_position ros_robot_controller_msgs/msg/ServosPosition '{"position": [{"id": 1, "position": 300}]}'
```

```
> ros2 topic pub /ros_robot_controller/bus_servo/set_position ros_robot_controller_msgs/msg/ServosPosition '{"position": [{"id": 1, "position": 300}]}'
```

可以得到三个可设置的参数，servo_id: 为舵机 ID，ID 为 1~6；

position: 为舵机的角度，范围为 0-1000；

这里将其设置为 1 号舵机旋转 300 角度。

```
> ros2 topic pub /ros_robot_controller/bus_servo/set_position ros_robot_controller_msgs/msg/ServosPosition '{"position": [{"id": 1, "position": 300}]}'
```

1.3 开启 APP 自启动服务

1) 玩法体验完成后，需开启 APP 服务（**若不开启则会影响后续 APP 玩法的使用**）。

在终端界面输入命令，按下回车，启动 APP 服务，接着稍等片刻即可开启。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

- 2) 启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

1.3.1 使用 python 文件控制

- 1) 在终端输入命令，启动舵机控制节点。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

- 2) 输入命令，启动玩法，可以看到机械臂的 4 号舵机在 200-500 角度来回旋转，并且在终端打印出舵机话题信息。

```
python3 ~/ros2_ws/src/example/example/simple/include/bus_servo_node.py
```

```
> python3 ~/ros2_ws/src/example/example/simple/include/bus_servo_node.py
[INFO] [1753289942.688134053] [servo_control_demo]: duration=0.5, id=4, position=200
[INFO] [1753289943.189427960] [servo_control_demo]: duration=0.5, id=4, position=500
[INFO] [1753289943.690878293] [servo_control_demo]: duration=0.5, id=4, position=200
[INFO] [1753289944.192166052] [servo_control_demo]: duration=0.5, id=4, position=500
[INFO] [1753289944.693458125] [servo_control_demo]: duration=0.5, id=4, position=200
[INFO] [1753289945.194783699] [servo_control_demo]: duration=0.5, id=4, position=500
```

1.3.2 启动 launch 文件控制

- 1) 在启动机械臂 SDK 文件的窗口下，按下“Ctrl+C”关闭启动文件。
- 2) 接着再输入命令，开启舵机控制（在 launch 文件中调用的是 bus_servo_node.py 文件）。

```
ros2 launch example bus_servo.launch.py
```

```
> ros2 launch example bus_servo.launch.py
```

- 3) 玩法启动后可以看到机械臂的 4 号舵机在 200-500 角度来回旋转，并且在终端打印出舵机话题信息。

```
[ros_robot_controller-1] [INFO] [1753290908.788631273] [ros_robot_controller]: start
[bus_servo-2] [INFO] [1753290909.021840458] [servo_control_demo]: duration=0.5, id=4, position=200
[bus_servo-2] [INFO] [1753290909.523185643] [servo_control_demo]: duration=0.5, id=4, position=500
```

- 4) 若要关闭玩法，按下“Ctrl+C”即可，若玩法关闭失败，可重复多次按下。

1.4 程序分析

1.4.1 launch 文件分析

launch 文件位于：

/home/ubuntu/ros2_ws/src/example/example/simple/bus_servo.launch.py

◆ generate_launch_description 函数

```
def generate_launch_description():
    compiled = os.environ['need_compile']

    if compiled == 'True':
        robot_controller_package_path = get_package_share_directory('ros_robot_controller')
    else:
        robot_controller_package_path = '/home/ubuntu/ros2_ws/src/driver/ros_robot_controller'

    robot_controller_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(robot_controller_package_path, 'launch/ros_robot_controller.launch.py')
        ])),
    )

    bus_servo_node = Node(
        package='example',
        executable='bus_servo',
        output='screen',
    )

    return LaunchDescription([
        robot_controller_launch,
        bus_servo_node
    ])
```

包含 ros_robot_controller 功能包中的 launch/ros_robot_controller.launch.py 启动文件，用于启动机器人控制器相关节点。定义 bus_servo_node 节点，启动 example 功能包中的 bus_servo 可执行文件。

◆ 主程序

```
if __name__ == '__main__':
    # 创建一个LaunchDescription对象
    ld = generate_launch_description()

    ls = LaunchService()
    ls.include_launch_description(ld)
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现启动整个系统的功能。

1.4.2 python 文件分析

python 文件位置位于: /home/ubuntu/ros2_ws/src/example/example/simple/include/bus_servo_node.py

◆ 导入功能包

```
import time
import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger
from ros_robot_controller_msgs.msg import ServoPosition, ServosPosition
```

① **ServoPosition, ServosPosition**: 这些是自定义的 ROS 消息类型，分别表示单个舵机的位置和多个舵机的位置。它们来自于 `ros_robot_controller_msgs.msg` 包。

◆ ServoController 类

```
class ServoController(Node):
    def __init__(self):
        super().__init__('servo_control_demo')
        self.pub = self.create_publisher(ServosPosition, '/ros_robot_controller/bus_servo/set_position', 1)

        #等待机械臂底层控制服务启动
        self.client = self.create_client(Trigger, '/ros_robot_controller/init_finish')
        self.client.wait_for_service()
```

初始化 ROS 节点（名称为 `servo_control_demo`），创建舵机位置发布者（话题为 `/ros_robot_controller/bus_servo/set_position`）；等待机器人控制器初始化完成服务（`/ros_robot_controller/init_finish`）启动，确保底层控制就绪。

◆ set_servo_position 类

```
def set_servo_position(self, duration, positions):
    msg = ServosPosition()
    msg.duration = float(duration)
    position_list = []
    for i in positions:
        position = ServoPosition()
        position.id = i[0]
        position.position = int(i[1])
        position_list.append(position)
    msg.position = position_list
    self.pub.publish(msg)
    for pos in position_list:
        self.get_logger().info(f'duration={msg.duration}, id={pos.id}, position={pos.position}')
```

构造舵机位置消息（`ServosPosition`），设置运动持续时间（`duration`）和目标位置列表（`positions`）：遍历输入的位置元组（格式为（舵机 ID，目标位置）），为每个舵

机构建 ServoPosition 消息，包含 ID 和位置信息。发布消息到舵机控制话题，并打印日志显示持续时间、舵机 ID 和目标位置。

◆ main 函数

```
def main(args=None):
    rclpy.init(args=args)
    controller = ServoController()

    try:
        while rclpy.ok():
            controller.set_servo_position(0.5, ((4, 200),)) # 设置舵机 ID 4 到位置 200
            time.sleep(0.5) # 等待 0.5 秒
            controller.set_servo_position(0.5, ((4, 500),)) # 设置舵机 ID 4 到位置 500
            time.sleep(0.5) # 等待 0.5 秒
    except KeyboardInterrupt:
        pass
    finally:
        controller.destroy_node() # 清理节点
        rclpy.shutdown() # 关闭 ROS 2
```

初始化 ROS 2, 创建 ServoController 节点实例。在循环中通过 set_servo_position 方法控制 4 号舵机在位置 200 和 500 之间交替运动，每次停留 0.5 秒。捕获键盘中断（KeyboardInterrupt），清理节点并关闭 ROS 2。

2. LED 控制节点

STM32 控制板板载了一个 LED，我们通过树莓派作为主控板，STM32 单片机进行通信，通过串口对 LED 进行控制。为了能够更加方便和规范的控制 LED，我们就需要设置一个 ROS 节点作为 LED 的接口。

2.1 LED 控制节点介绍

2.1.1 准备工作

1) 根据“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意事项”内容安装好机械臂平台。

2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。

3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

2.1.2 启动 LED 控制节点

1) 单击 ，启动命令栏，输入指令，按下回车（密码输入时不显示，输入完成后按下回车即可），关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

2) 输入命令，启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

3) 打开一个新的命令栏，输入命令，查看机械臂节点启动后的话题。

```
ros2 topic list
```

```
/ros_robot_controller/bus_servo/set_position  
/ros_robot_controller/bus_servo/set_state  
/ros_robot_controller/button  
/ros_robot_controller/enable_reception  
/ros_robot_controller/imu_raw  
/ros_robot_controller/joy  
/ros_robot_controller/pwm_servo/set_state  
/ros_robot_controller/sbus  
/ros_robot_controller/set_buzzer  
/ros_robot_controller/set_conveyor_speed  
/ros_robot_controller/set_led  
/ros_robot_controller/set_motor_speeds  
/ros_robot_controller/set_oled  
/ros_robot_controller/set_stepper_subdivision
```

话题	功能说明
/ros_robot_controller/set_led	控制 LED 灯的亮灭

2.2 通过话题控制 LED

2.2.1 在终端控制

1) 打开一个新的命令栏，输入命令

```
ros2 topic pub --once /ros_robot_controller/set_led ros_robot_controller
```

```
_msgs/msg/LedState "{id: 2,on_time: 0.1,off_time: 0.2,repeat: 10}"
```

```
> ros2 topic pub --once /ros_robot_controller/set_led ros_robot_controller_msgs/msg/LedState  
"{id: 2,on_time: 0.1,off_time: 0.2,repeat: 10}"
```

可以得到四个可设置的参数：

- ① “id” 为 LED 的 ID，我们 32 主控板上板载的 LED 灯 ID 为 2；
- ② “on_time” 为 LED 亮的时间，单位为 s；
- ③ “off_time” 为 LED 灭的时间，单位为 s；
- ④ “repeat” 为 LED 亮、灭重复的次数。

2) 参数设置完成后按下回车，即可运行程序。

```
> ros2 topic pub --once /ros_robot_controller/set_led ros_robot_controller_msgs/msg/LedState  
"{id: 2,on_time: 0.1,off_time: 0.2,repeat: 10}"  
publisher: beginning loop  
publishing #1: ros_robot_controller_msgs.msg.LedState(id=2, on_time=0.1, off_time=0.2, repeat  
=10)
```

3) 运行完毕后，按下 “Ctrl+C” 即可退出，若退出失败，可重复多次按下。

2.2.2 使用 Python 文件控制

1) 输入命令，启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

2) 输入命令启动玩法，可以看到 32 控制板板载的 LED 灯闪烁蓝灯十次后停止，并且在终端打印出 LED 参数信息。

```
python3 ~/ros2_ws/src/example/example/simple/include/led_node.py
```

```
> python3 ~/ros2_ws/src/example/example/simple/include/led_node.py  
[INFO] [1753292707.267960683] [led_controller]: Published LED State: id=2, on_time=0.1, off_t  
ime=0.5, repeat=10
```

2.2.3 启动 launch 文件控制

1) 在启动机械臂 SDK 文件的窗口下，按下 “Ctrl+C” 关闭启动文件。

2) 接着再输入命令，开启舵机控制（在 launch 文件中调用的是 led_node.py 文件）。

```
ros2 launch example led.launch.py
```

```
> ros2 launch example led.launch.py
```

3) 玩法启动后可以看到 32 控制板板载的 LED 灯闪烁蓝灯十次后停止，并且在终端打印出 LED 参数信息。

```
[ros_robot_controller-1] [INFO] [1753292792.505416409] [ros_robot_controller]: start  
[led-2] [INFO] [1753292792.540649280] [led_controller]: Published LED State: id=2, on_time=0.1, off_time=0.5, repeat=10
```

4) 若要关闭玩法，按下“Ctrl+C”即可，若玩法关闭失败，可重复多次按下。

2.3 开启 APP 自启动服务

1) 玩法体验完成后，需开启 APP 服务（**若不开启则会影响后续 APP 玩法的使用**）。

在终端界面输入命令，按下回车，启动 APP 服务，接着稍等片刻即可开启。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

2) 启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

2.4 程序分析

2.4.1 launch 文件分析

launch 文件位于：

/home/ubuntu/ros2_ws/src/example/example/simple/led.launch.py

◆ generate_launch_description 函数

```
def generate_launch_description():  
    compiled = os.environ['need_compile']  
  
    if compiled == 'True':  
        robot_controller_package_path = get_package_share_directory('ros_robot_controller')  
    else:  
        robot_controller_package_path = '/home/ubuntu/ros2_ws/src/driver/ros_robot_controller'  
  
    robot_controller_launch = IncludeLaunchDescription(  
        PythonLaunchDescriptionSource([os.path.join(robot_controller_package_path, 'launch/ros_robot_controller.launch.py')  
        ]),  
    )  
  
    led_node = Node(  
        package='example',  
        executable='led',  
        output='screen',  
    )  
  
    return LaunchDescription([  
        robot_controller_launch,  
        led_node  
    ])
```

包含 ros_robot_controller 功能包中的 launch/ros_robot_controller.launch.py 启动文件，用于启动机器人控制器相关节点。定义 led_node 节点，启动 example 功能

包中的 led 可执行文件，输出日志到屏幕。

◆ 主程序

```
if __name__ == '__main__':  
    # 创建一个LaunchDescription对象  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现启动整个系统的功能。

2.4.2 python 文件分析

python 文件位置位于：/home/ubuntu/ros2_ws/src/example/example/simple/include/led_node.py

◆ 导入功能包

```
import rclpy  
from rclpy.node import Node  
from std_srvs.srv import Trigger  
from ros_robot_controller_msgs.msg import LedState
```

LedState: 自定义的消息类型，用于描述 LED 灯的状态。

◆ LedController 类

```
class LedController(Node):  
    def __init__(self):  
        super().__init__('led_controller')  
        self.publisher = self.create_publisher(LedState, '/ros_robot_controller/set_led', 10)  
  
        #等待机械臂底层控制服务启动  
        self.client = self.create_client(Trigger, '/ros_robot_controller/init_finish')  
        self.client.wait_for_service()  
  
        # 发布消息  
        self.publish_led_state()
```

初始化 ROS 节点（名称为 led_controller），创建 LED 状态发布者（话题为

/ros_robot_controller/set_led)；等待机器人控制器初始化完成服务 (/ros_robot_controller/init_finish) 启动，确保底层控制就绪；调用 publish_led_state 方法发布 LED 控制消息。

◆ publish_led_state 函数

```
def publish_led_state(self):
    msg = LedState()
    msg.id = 2
    msg.on_time = 0.1
    msg.off_time = 0.5
    msg.repeat = 10

    self.publisher.publish(msg)
    self.get_logger().info(f'Published LED State: id={msg.id}, on_time={msg.on_time}, off_time={msg.off_time}, repeat={msg.repeat}')
```

构造 LED 状态消息 (LedState)，设置目标 LED 的 ID (id=2)、点亮时间 (on_time=0.1 秒)、熄灭时间 (off_time=0.5 秒) 和重复次数 (repeat=10 次)；发布消息到 LED 控制话题，并打印日志显示 LED 的控制参数。

◆ main 函数：

```
def main(args=None):
    rclpy.init(args=args)
    led_controller = LedController()

    rclpy.spin_once(led_controller) # 发布一次
    rclpy.shutdown()
```

初始化 ROS 2，创建 LedController 节点实例。通过 rclpy.spin_once 执行一次节点回调，完成 LED 状态消息的发布。关闭 ROS 2，结束程序。

3. 蜂鸣器控制节点

STM32 控制板板载了一个蜂鸣器，我们通过树莓派作为主控板，STM32 单片机进行通信，通过串口对蜂鸣器进行控制。为了能够更加方便和规范的控制蜂鸣器，我们就需要设置一个 ROS 节点作为蜂鸣器的接口。

3.1 蜂鸣器控制节点介绍

3.1.1 准备工作

- 1) 根据“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意事项”内容安装好机械臂平台。
- 2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。
- 3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

3.1.2 启动蜂鸣器控制节点

- 1) 单击，启动命令栏，输入指令，按下回车（密码输入时不显示，输入完成后按下回车即可），关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

- 2) 输入命令启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

- 3) 打开一个新的命令栏，输入命令查看机械臂节点启动后的话题。

```
ros2 topic list
```

```
/ros_robot_controller/joy
/ros_robot_controller/pwm_servo/set_state
/ros_robot_controller/sbus
/ros_robot_controller/set_buzzer
/ros_robot_controller/set_led
/ros_robot_controller/set_motor
/ros_robot_controller/set_oled
/rosout
/servo_controller
```

话题	功能说明
----	------

/ros_robot_controller/set_buzzer	控制蜂鸣器的开关
----------------------------------	----------

3.2 通过话题控制蜂鸣器

3.2.1 在终端控制

1) 打开一个新的命令栏，输入命令

```
ros2 topic pub --once /ros_robot_controller/set_buzzer ros_robot_controller_msgs/msg/BuzzerState "{freq: 1900, on_time: 0.1, off_time: 0.2, repeat: 10}"
```

```
> ros2 topic pub --once /ros_robot_controller/set_buzzer ros_robot_controller_msgs/msg/BuzzerState "{freq: 1900, on_time: 0.1, off_time: 0.2, repeat: 10}"
```

可以得到四个可设置的参数。

- ① freq: 为蜂鸣器的频率，范围为 2~4KHZ，频率越大音调越高；
- ② on_time: 为蜂鸣器响的时间，单位为 s；
- ③ off_time: 为蜂鸣器停止发声的时间，单位为 s；
- ④ repeat: 为蜂鸣器重复的发声和停止重复的次数。

2) 参数设置完成后按下回车，即可运行程序。

```
> ros2 topic pub --once /ros_robot_controller/set_buzzer ros_robot_controller_msgs/msg/BuzzerState "{freq: 1900, on_time: 0.1, off_time: 0.2, repeat: 10}"
```

3) 运行完毕后，按下“Ctrl+C”即可退出，若退出失败，可重复多次按下。

3.2.2 使用 Python 文件控制

◆ 输入命令启动玩法，可以听到蜂鸣器“滴”十次之后停止，并且在终端打印出蜂鸣器参数信息。

```
python3 ~/ros2_ws/src/example/example/simple/include/buzzer_node.py
```

```
> python3 ~/ros2_ws/src/example/example/simple/include/buzzer_node.py  
[INFO] [1753872276.613559060] [buzzer_controller]: Published BuzzerState: freq=1500, on_time=0.1, off_time=0.5, repeat=10
```

3.2.3 启动 launch 文件控制

- 1) 在启动机械臂 SDK 文件的窗口下，按下“Ctrl+C”关闭启动文件。
- 2) 接着再输入命令开启舵机控制（在 launch 文件中调用的是 buzzer.py 文件）。

```
ros2 launch example buzzer.launch.py
> ros2 launch example buzzer.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2025-07-30-10-46-49-531073-raspberrypi-4146
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros_robot_controller-1]: process started with pid [4147]
[INFO] [buzzer-2]: process started with pid [4149]
[ros_robot_controller-1] [INFO] [1753872410.699753547] [ros_robot_controller]
: start
[buzzer-2] [INFO] [1753872410.737918447] [buzzer_controller]: Published BuzzerState: freq=1500, on_time=0.1, off_time=0.5, repeat=10
```

- 3) 玩法启动后可以听到 32 驱动板板载的蜂鸣器“滴”十次后停止，并且在终端打印出蜂鸣器参数信息。
- 4) 若要关闭玩法，按下“Ctrl+C”即可，若玩法关闭失败，可重复多次按下。

3.3 开启 APP 自启动服务

- 1) 玩法体验完成后，需开启 APP 服务（**若不开启则会影响后续 APP 玩法的使用**）
- 2) 在终端界面输入命令，按下回车，启动 APP 服务，接着稍等片刻即可开启。

```
ros2 launch bringup bringup.launch.py
> ros2 launch bringup bringup.launch.py
```

- 3) 启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

3.4 程序分析

3.4.1 launch 文件分析

launch 文件位于：

/home/ubuntu/ros2_ws/src/example/example/simple/buzzer.launch.py

◆ generate_launch_description 函数

```
def generate_launch_description():
    compiled = os.environ['need_compile']

    if compiled == 'True':
        robot_controller_package_path = get_package_share_directory('ros_robot_controller')
    else:
        robot_controller_package_path = '/home/ubuntu/ros2_ws/src/driver/ros_robot_controller'

    robot_controller_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(robot_controller_package_path, 'launch/ros_robot_controller.launch.py')
        ]),
    )

    buzzer_node = Node(
        package='example',
        executable='buzzer',
        output='screen',
    )

    return LaunchDescription([
        robot_controller_launch,
        buzzer_node
    ])
```

包含 `ros_robot_controller` 功能包中的 `launch/ros_robot_controller.launch.py` 启动文件，用于启动机器人控制器相关节点（提供底层硬件控制支持）。定义 `buzzer_node` 节点，启动 `example` 功能包中的 `buzzer` 可执行文件，输出日志到屏幕，用于控制蜂鸣器工作。

◆ 主函数：

```
if __name__ == '__main__':
    # 创建一个LaunchDescription对象
    ld = generate_launch_description()

    ls = LaunchService()
    ls.include_launch_description(ld)
    ls.run()
```

创建 `LaunchDescription` 对象和 `LaunchService` 服务，将启动描述添加到启动服务并运行，实现启动整个系统的功能。

3.4.2 python 文件分析

python 文件位置位于：`/home/ubuntu/ros2_ws/src/example/example/simple/include/buzzer_node.py`

◆ 导入功能包

```
import time
import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger
from ros_robot_controller_msgs.msg import BuzzerState
```

BuzzerState: 自定义的消息类型，用于设置蜂鸣器的状态。

◆ BuzzerController 类

```
class BuzzerController(Node):
    def __init__(self):
        super().__init__('buzzer_controller')
        self.pub = self.create_publisher(BuzzerState, '/ros_robot_controller/set_buzzer', 1)

        #等待机械臂底层控制服务启动
        self.client = self.create_client(Trigger, '/ros_robot_controller/init_finish')
        self.client.wait_for_service()

    def set_buzzer(self, freq, on_time, off_time, repeat):
        msg = BuzzerState()
        msg.freq = freq
        msg.on_time = on_time
        msg.off_time = off_time
        msg.repeat = repeat

        # 发布消息
        self.pub.publish(msg)
        self.get_logger().info(f'Published BuzzerState: freq={msg.freq}, on_time={msg.on_time}, off_time={msg.off_time}, repeat={msg.repeat}')
```

初始化 ROS 节点（名称为 `buzzer_controller`），创建蜂鸣器状态发布者（话题为 `/ros_robot_controller/set_buzzer`）；等待机器人控制器初始化完成服务（`/ros_robot_controller/init_finish`）启动，确保底层控制就绪。

◆ set_buzzer 函数

```
def set_buzzer(self, freq, on_time, off_time, repeat):
    msg = BuzzerState()
    msg.freq = freq
    msg.on_time = on_time
    msg.off_time = off_time
    msg.repeat = repeat

    # 发布消息
    self.pub.publish(msg)
    self.get_logger().info(f'Published BuzzerState: freq={msg.freq}, on_time={msg.on_time}, off_time={msg.off_time}, repeat={msg.repeat}')
```

构造蜂鸣器状态消息（`BuzzerState`），设置蜂鸣器的频率（`freq`）、发声时间（`on_time`）、停止时间（`off_time`）和重复次数（`repeat`）；发布消息到蜂鸣器控制话题，并打印日志显示蜂鸣器的控制参数。

◆ main 函数

```
def main(args=None):  
    rclpy.init(args=args)  
    controller = BuzzerController()  
  
    # 发送蜂鸣器状态  
    controller.set_buzzer(freq=1500, on_time=0.1, off_time=0.5, repeat=10)  
    time.sleep(5)  
    controller.destroy_node() # 清理节点  
    rclpy.shutdown() # 关闭 ROS 2
```

初始化 ROS 2，创建 BuzzerController 节点实例。调用 set_buzzer 方法控制蜂鸣器以 1500Hz 频率、0.1 秒发声 / 0.5 秒停止的周期重复工作 10 次。等待 5 秒（确保蜂鸣器完成指定动作），清理节点并关闭 ROS 2。

4. 上位机控制

4.1 上位机介绍

上位机与下位机对应，用来给下位机发送指令以及从下位机接收反馈数据。一般作为上位机的就是电脑，通过电脑上运行的软件（即上位机软件）来控制下位机。

本节将介绍 ArmPi Ultra 上位机软件的界面和功能。

4.1.1 通过图标启动上位机

双击桌面上的 Arm 上位机图标 ，即可启动上位机。

4.1.2 通过命令行启动上位机

1) 打开一个新的命令栏，输入命令进入源码文件目录下。

```
cd software/armpi_ultra_control
```

```
> cd software/armpi_ultra_control
```

2) 输入命令打开上位机。

```
python3 main.py
```

```
> python3 main.py
```

4.1.3 上位机界面分布

上位机界面如下图所示，分别为①舵机操控区域、②动作详情列表、③动作组设置区域、④偏差设置区域，⑤坐标设置区域等5个区域：



下列为各个区域的功能介绍：

①：舵机操控区域

舵机操控区域显示了被选中的舵机图标，通过调整对应滑杆值即可调整舵机位置。

图标	功能说明
	代表舵机的 ID 号，这里以 1 为例。
	用于调整舵机位置，最小值为 0，最大值为 1000。
	用于调整舵机偏差，最小值为 -125，最大值为 125（关于偏差调节可前往“ 偏差调节 ”进行学习）。

②：动作详情列表

动作详情列表显示了当前动作的编号、执行时间，以及里各个舵机的位置信息。

	编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
	1	1200	400	500	250	935	800	500
	2	200	400	500	250	935	800	500
	3	500	540	500	250	935	800	500
	4	700	540	500	250	935	800	500
	5	700	540	500	250	935	800	300
	6	1200	540	500	250	935	500	300
	7	200	540	500	250	935	500	300
	8	500	400	500	250	935	500	300

图标	功能说明
	动作编号。
	执行该动作所用的时间，单位为 mm。
	该 ID 下对应的舵机数值，双击下方数值 500 可直接修改。

③：动作组设置区域

动作组设置区域可对动作进行增加、删除、更新、插入、运行、打开、串联等操作。

图标	功能说明
	运行单个动作所需的时间，点击 1000 可进行修改。
	动作组运行所需的总时长。
	点击后机器人舵机会掉电，关节也会变得松弛，此时可以用手掰动机器人，设计动作（ 若掰不动不要硬掰，重新点击即可 ）。
	将手掰舵机的位置信息读取出来。（需要搭配“手掰编程”按钮使用）。
	将当前舵机操控区域的各个舵机数值作为一个动作，添加至动作详情列表的最后一行。

删除动作 删除全部	删除动作：删除动作详情列表里选中的动作。 删除全部：删除动作详情列表里所有动作。
更新动作	将动作详情列表中选中的数值进行替换。 （舵机数值替换为舵机操控区域的当前舵机值，动作运行时间替换为“动作时间”内设定的时间）
插入动作	在选中的动作上面插入一行动作。 （其中动作的时间为“动作时间(ms)”的时间，角度数值为舵机操控区域的舵机值）
上移动作	将选中的动作向上移动一行。
下移动作	将选中的动作向下移动一行。
☐ 循环 运行	单击该按钮后将会运行一次动作详情列表的动作。 （如果勾选了“循环”，机器人则重复运行动作）
打开动作文件	单击后选择要打开的动作组即可将动作组数据加载到动作详情列表中。 （动作组文件路径： /home/ubuntu/software/armpi_ultra_control/Action Groups）
保存动作文件	将此时动作详情列表里的动作保存到指定位置。 （动作组文件路径： /home/ubuntu/software/armpi_ultra_control/Action Groups）

	打开一个动作组后，点击串联动作文件按钮，继续打开另一个动作组文件，可将两个动作组文件串联成一个新的动作组。
动作组 	可在上位机中显示已保存的动作组。
	执行一次动作组选项框中的动作组。
	停止正在运行的动作组。
	退出当前上位机操作界面。
	按下此按钮，动作组选择栏即可刷新。

④：偏差设置区域（此区域功能按键了解即可，关于偏差调节可前往“[偏差调节](#)”）

图标	功能说明
	单击后即可自动读取保存的偏差。
	单击后即可将通过上位机调整好的偏差下载到机器人中。
	单击后即可将舵机操控区域的所有舵机恢复至 500 位置。

⑤：坐标设置区域

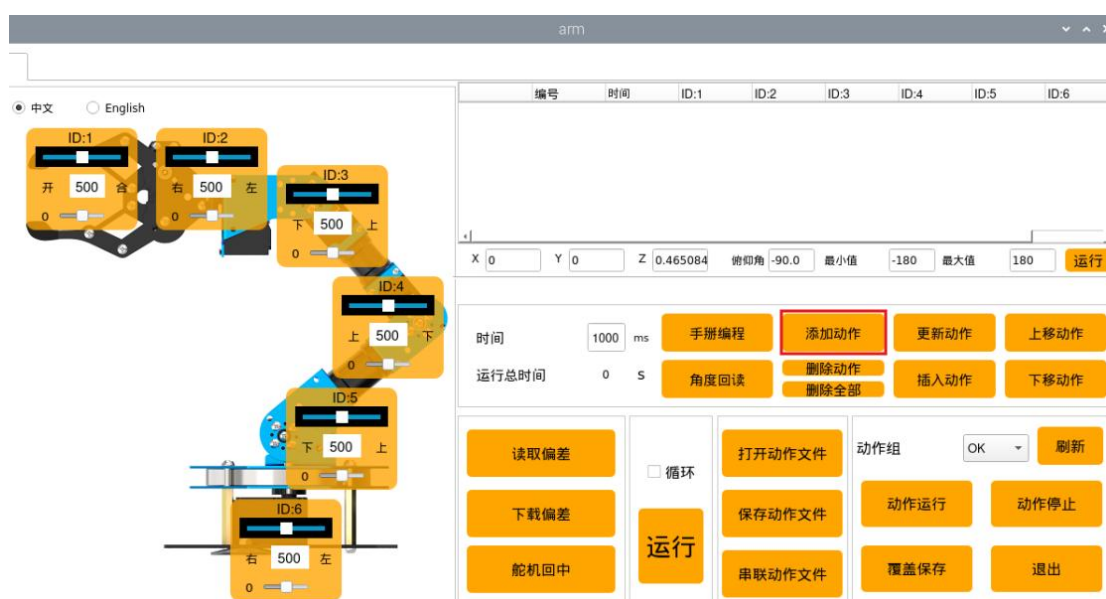
通过输入数值来设置机械臂的位置，可控制机械臂以一个整体沿三轴坐标（X、Y、Z）的轨迹以及结合俯仰角进行运动。

4.2 动作组编辑及调用

4.2.1 动作组编辑

点击桌面上的 Arm 上位机图标 ，打开上位机。

步骤 1： 首先我们设置一下机械臂的初始姿态，先点击舵机回中，接着按照下图所示拖动舵机滑杆按钮。然后点击界面右侧的“**添加动作**”按钮，将动作添加至动作详情列表里。



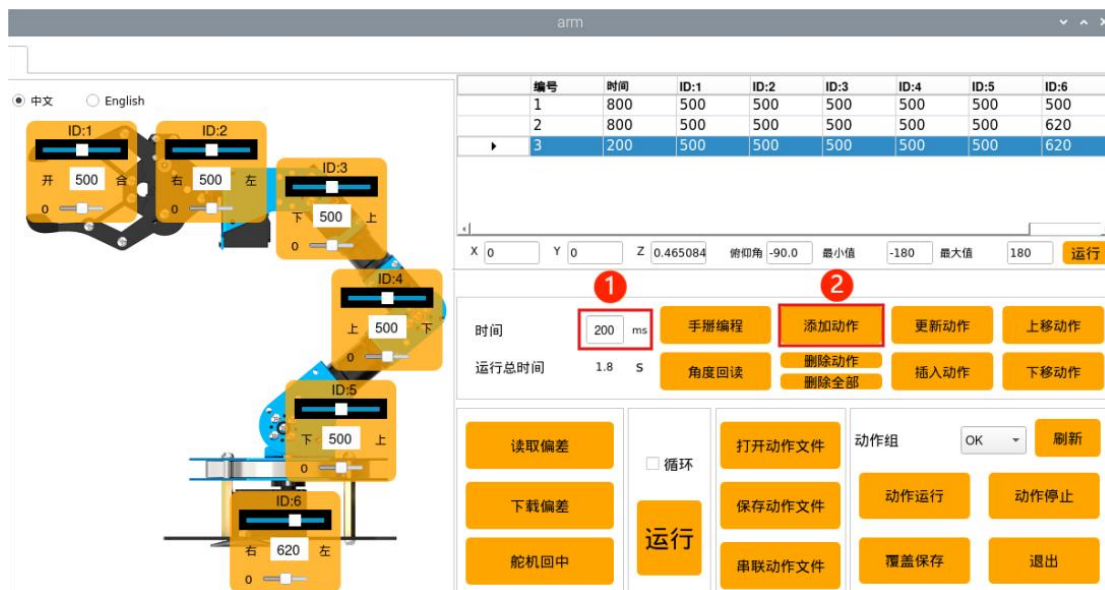
步骤 2： 动作时间我们设置为800ms，将鼠标光标移动到动作时间框填入**800**，然后点击“**更新动作**”即可同步时间。



步骤 3: 接下来我们就让机械臂进行左移，在左侧的操控区域，将ID为6号舵机的滑杆按照下图的数值进行滑动。然后点击“添加动作”，将这个动作添加至右侧的动作列表中，动作时间同样设置为800ms。



步骤 4: 我们添加一组过渡动作，让动作协调更加自然一些。选择编号为2的动作，设置动作运行时间200ms，点击“添加动作”即可。



步骤 5: 接着我们就将机械臂右移，在左侧的操控区域，将 ID 为 6 号舵机的滑杆按照下图的数值进行滑动。然后点击“添加动作”，将这个动作添加至右侧的动作列表中，动作时间同样设置为 800ms。



步骤 6: 同理，为了动作更加顺畅，我们将第5号动作作为过渡。选中第4组动作后，点击“添加动作”，动作时间设置为200ms。

	编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
	1	800	500	500	500	500	500	500
	2	800	500	500	500	500	500	620
	3	200	500	500	500	500	500	620
	4	800	500	500	500	500	500	420
▶	5	200	500	500	500	500	500	420

步骤 7: 接下来我们就让机械臂的夹子开合，调整 1 号舵机滑杆按钮，并且动作时间设定为 400ms 后添加为第 6 组动作。



编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
1	800	500	500	500	500	500	500
2	800	500	500	500	500	500	620
3	200	500	500	500	500	500	620
4	800	500	500	500	500	500	420
5	200	500	500	500	500	500	420
6	400	580	500	500	500	500	420

步骤 8: 第 7 组动作作为第 6 组动作的过渡，动作时间设置为 200ms 即可。

	编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
	1	800	500	500	500	500	500	500
	2	800	500	500	500	500	500	620
	3	200	500	500	500	500	500	620
	4	800	500	500	500	500	500	420
	5	200	500	500	500	500	500	420
	6	400	580	500	500	500	500	420
▶	7	200	580	500	500	500	500	420

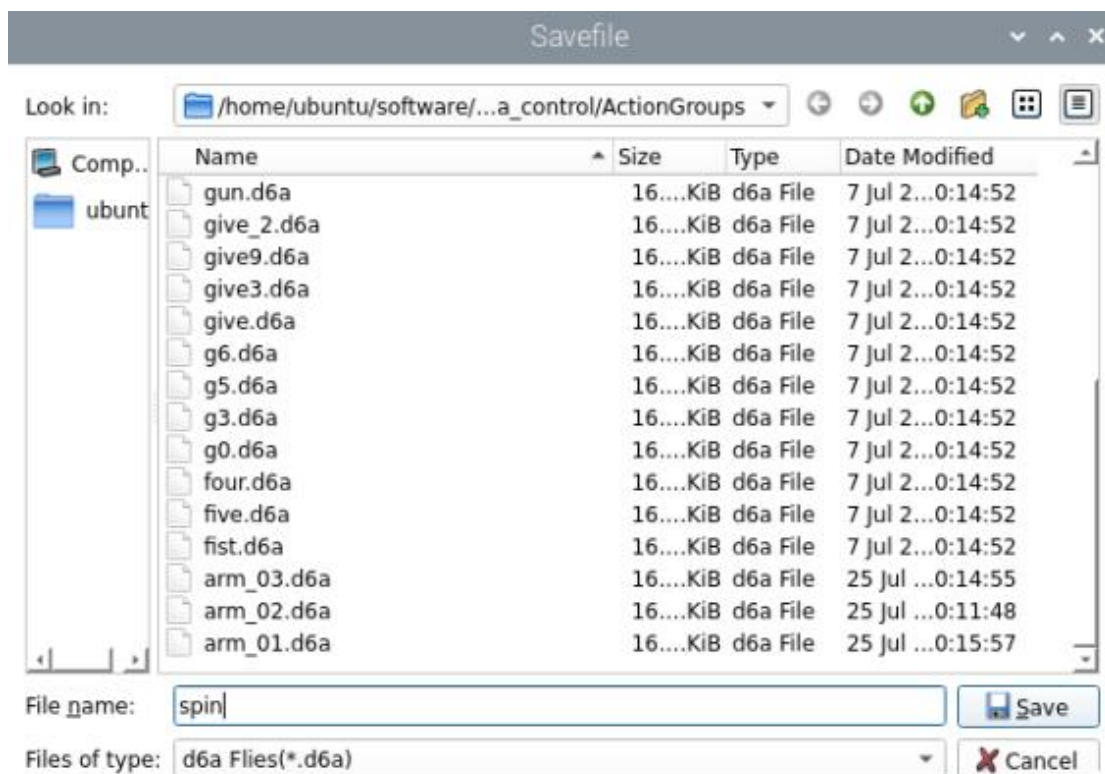
步骤 9: 接着编写8号动作调整爪子即可。

	编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
	1	800	500	500	500	500	500	500
	2	800	500	500	500	500	500	620
	3	200	500	500	500	500	500	620
	4	800	500	500	500	500	500	420
	5	200	500	500	500	500	500	420
	6	400	580	500	500	500	500	420
	7	200	580	500	500	500	500	420
▶	8	400	400	500	500	500	500	420

4.2.2 动作保存

为了便于后期的调试和管理，可将动作保存起来。单击“保存动作文件”按钮，在打开的界面中路径选择为

“/home/ubuntu/software/armpi_ultra_control/ActionGroups”，这里以“spin”为例进行命名，单击“Save”即可（注意：命名时尽量使用英文，若使用中文可能导致动作调用失败）。



4.2.3 动作组调用

1) 打开一个新的命令栏，输入命令进入源码文件目录下。

```
cd software/armpi_ultra_control
```

```
> cd software/armpi_ultra_control
```

2) 输入命令打开源码文件。

```
vim control_demo.py
```

```
> vim control_demo.py
```

3) 在红框内填入动作组的名称（注意：动作组需要保存在当前路径的 ActionGroups 下，红框内的动作组名称不包括后缀，以字符形式传入）。

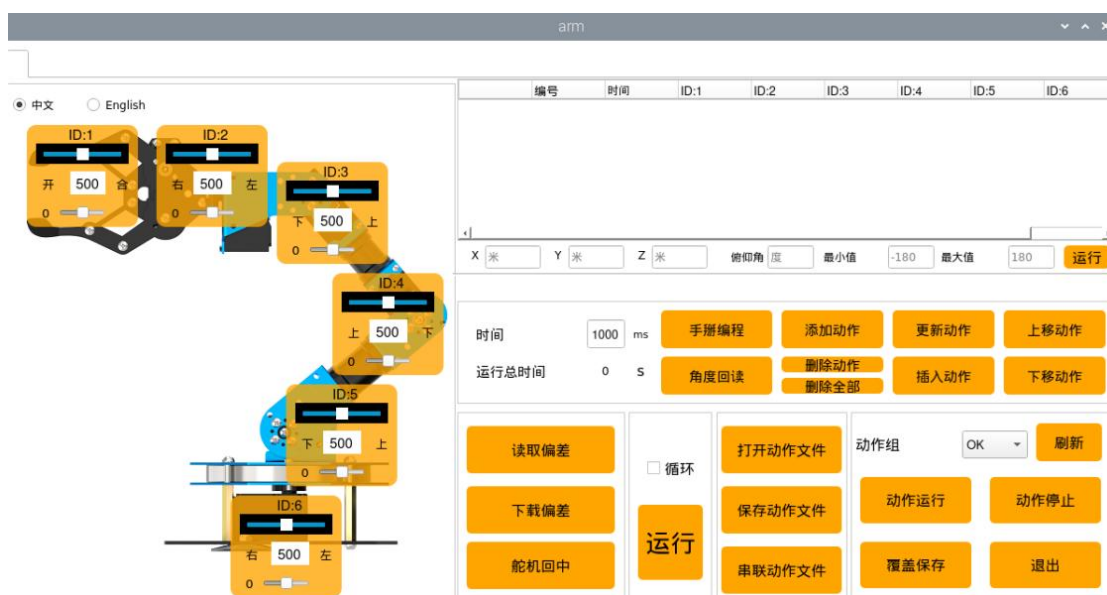
```
# 动作组需要保存在当前路径的ActionGroups下  
controller.runAction('five') # 参数为动作组的名称，不包含>  
后缀，以字符形式传入
```

4) 输入指令“python3 control_demo.py”运行程序，即可调用设定好的动作组。

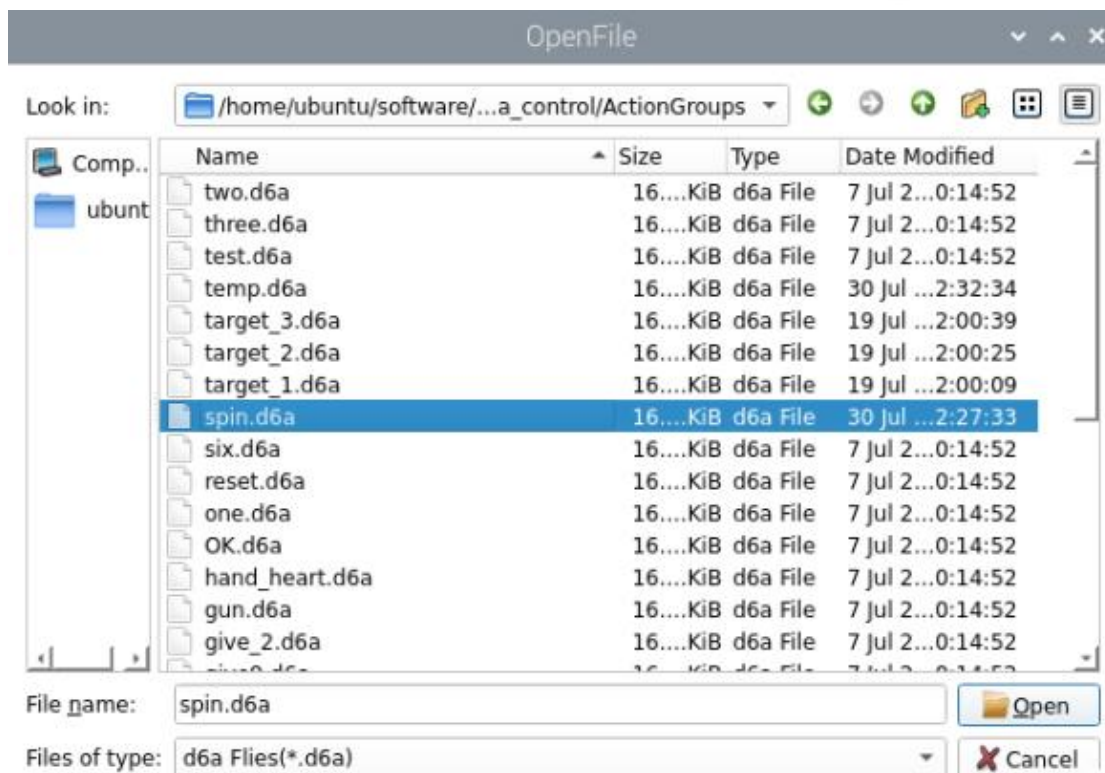
```
> python3 control_demo.py
```

4.3 串联动作组

步骤 1：远程连接系统后，启动 Arm 上位机。



步骤 2：点击动作组设置区的“串联动作文件”按钮，进入目录“/home/ubuntu/software/armpi_ultra_control/ActionGroups”，选择“spin.d6a”，双击打开。



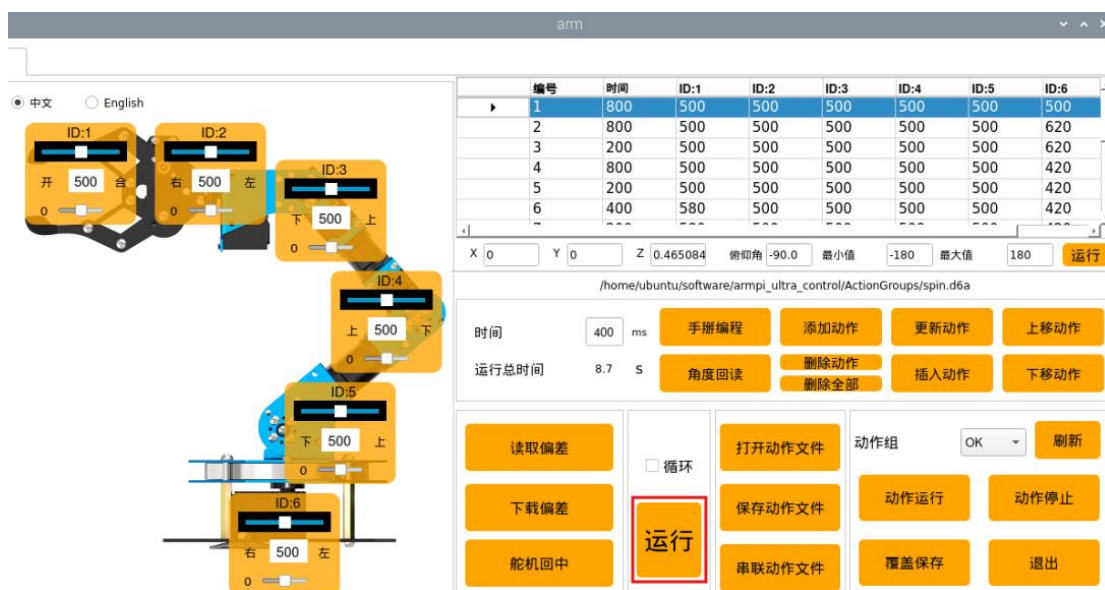
步骤 3: 此时，我们看到动作组详情列表中显示了“spin”动作组参数。



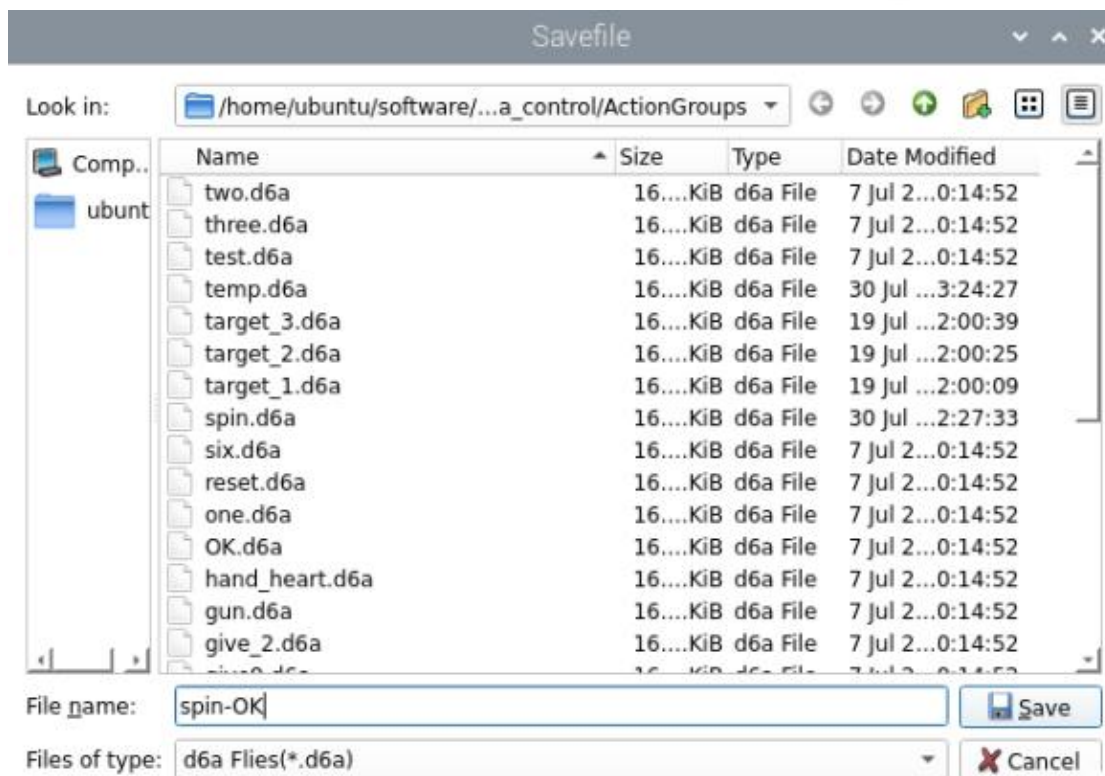
步骤 4: 再次点击“串联动作文件”，选择“OK.d6a”，双击打开。我们可以看到“OK”动作组文件已经串联进来了。

	编号	时间	ID:1	ID:2	ID:3	ID:4	ID:5	ID:6
	7	200	580	500	500	500	500	420
	8	400	400	500	500	500	500	420
	9	1500	200	500	240	900	750	500
	10	200	200	500	240	900	750	500
	11	800	650	500	240	900	750	500
	12	800	200	500	240	900	750	500
	13	800	650	500	240	900	750	500
	14	800	200	500	240	900	750	500

步骤 5: 将光标移动至编号 1，然后点击“运行”，将串联的新的动作组在线运行一遍。



步骤6: 点击“保存动作文件”按钮，可将新串联的动作组进行保存，以便后期调试（**注意：命名时使用英文，若使用中文可能导致动作调用失败**）。



4.4 偏差调节

机器人在使用过程中，若因更换舵机或者其他操作使舵机主轴偏移产生偏差，都需要进行偏差调试，而偏差分为小偏差和大偏差两种情况。根据情况不同，调试的方法也有所不同。

4.4.1 准备工作

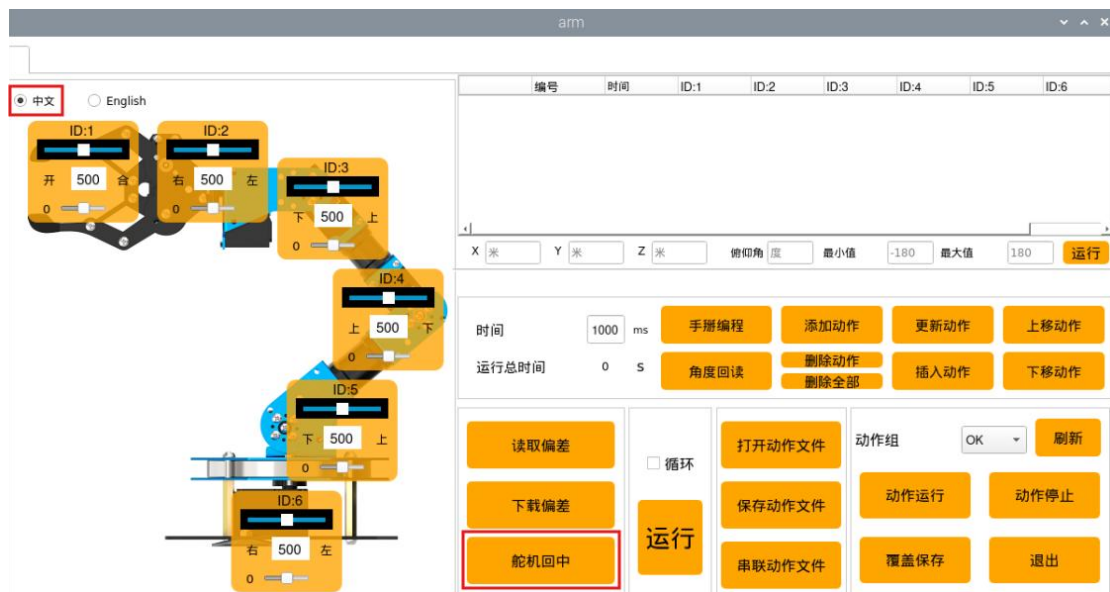
- 1) 根据“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意事项”内容安装好机械臂平台。
- 2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。
- 3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

4.4.2 判断大小偏差



- 1) 远程连接机器人之后，点击系统桌面的上位机图标。
- 2) 进入上位机界面，点击左上角语音选项，将其切换为中文。然后在上位机界面点击

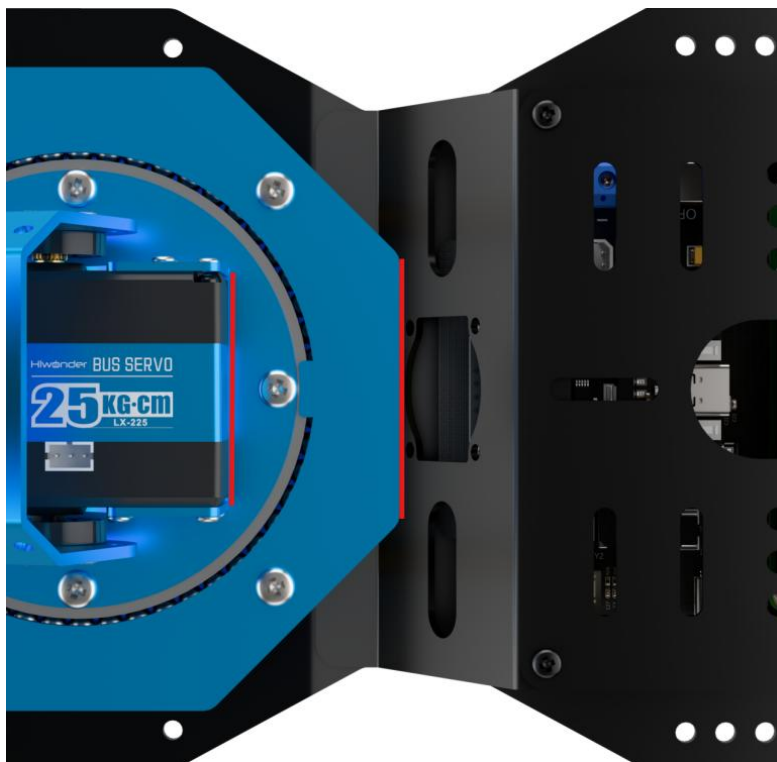
“舵机回中”，让机械臂笔直立起。



3) 如果舵机复位后，机械臂 3~5 号舵机的主轴螺丝呈笔直的一条线（下文以中心线代称），如下图所示：



4) 6 号舵机与金属外壳平行，如下图所示：



5) 2号舵机上方的金属外壳与相机外壳两条直线呈平行状态；1号舵机的机械爪处于轻微张开的状态，如下图；则表明机械臂无需调整偏差，可跳过本节内容。



6) 若舵机与中心线夹角小于 30° ，通过上位机调节可以归位，则为小偏差，可以通过“[小偏差调节](#)”来进行调节；偏差情况以3号舵机为例，如下图所示：



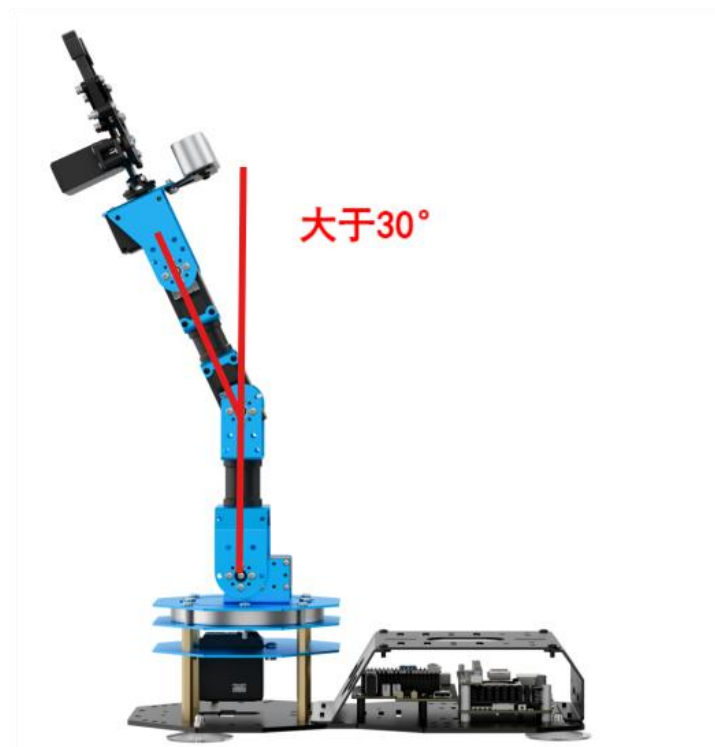
注意：偏差调节的方法可以观看本节课对应视频进行学习。

这里的舵机与中心线夹角是通过计算得到的：

上位机调节舵机的滑杆范围是 0~1000，而舵机上电的可转动范围是 0~240 度， $240/1000=0.24$ 度，即是滑杆的一个单位对应舵机转动角度 0.24；

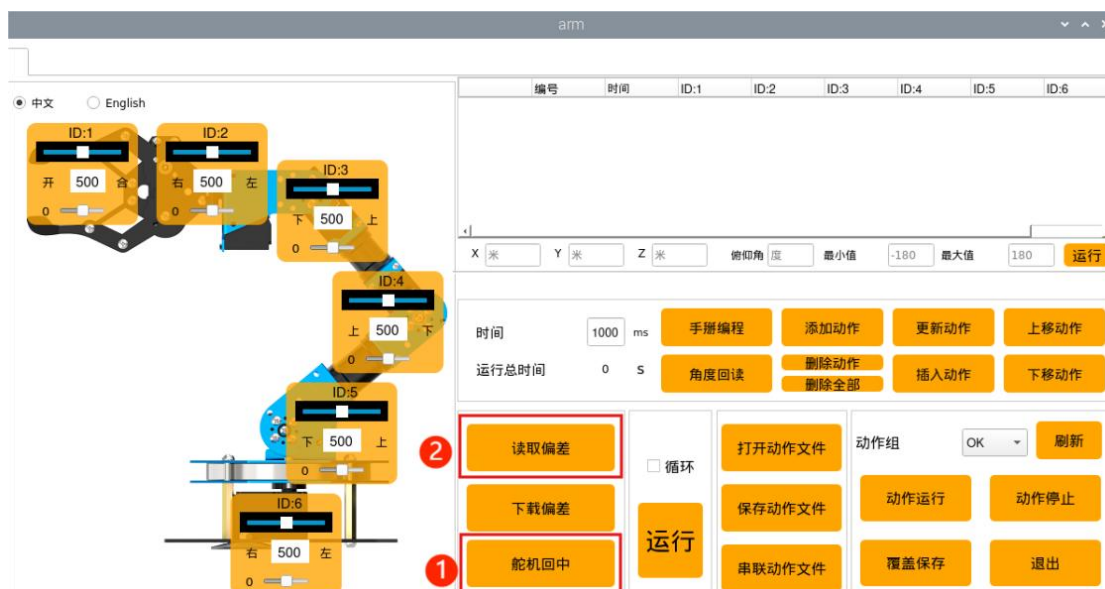
而调节小滑杆的范围是 ± 125 ，换算成小偏差的调节范围则是 $125 \times 0.24 = 30^\circ$ 。

7) 如舵机与中心线的夹角大于 30° ，通过上位机调节到极限数值 (± 125) 也无法归位，则为大偏差，可以通过“[大偏差调节](#)”来进行调节；偏差情况以 3 号舵机为例，如下图所示：



4.4.3 小偏差调节

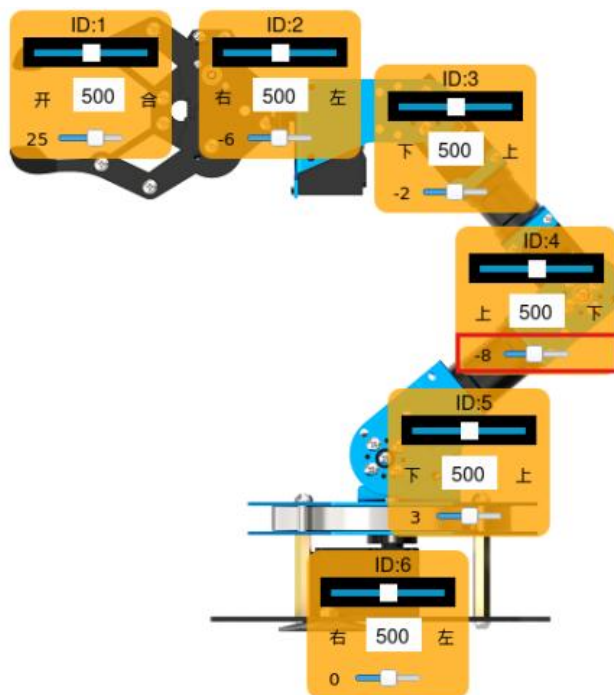
1) 打开上位机，点击“舵机回中”等待机械臂回到中位姿态后，点击“读取偏差”选项，这样机械臂上的偏差数据就会读取至上位机中。



2) 这里以 ID4 号为例，那么在读取偏差之后，就可以拖动上位机中 ID4 号下方的滑杆来进行偏差调节了。

注意：小偏差的调节可直接在上位机的界面拖动舵机的偏差滑杆来调节(注意：**调节偏差的是小滑杆**)

3) 拖动 4 号舵机的小滑杆，使机械臂呈直线（下图所示数值仅为参考，用户需调节目标，结合实际情况进行调节）



4) 接着点击“**下载偏差**”，将调节好的偏差数据保存到机械臂中。



4.4.4 大偏差调节

大偏差的调节则需要将舵机从 U 型支架上拆下来，进行中位后，安装，安装完成后再重复小偏差调节的步骤。

1) 在上位机界面点击“**舵机回中**”，让机械臂笔直立起。



2) 此时可以发现 2 号舵机有明显的偏差，偏差角度大于 30° 。



3) 将机械臂断电（以下涉及拆卸步骤，切勿在通电下进行）。

4) 将 2 号舵机主舵盘的所有螺丝和副舵盘上的主轴螺丝拧下（先拧外围的四颗舵盘螺丝，再拧主轴螺丝）。



5) 再将 U 型支架拆下。



6) 拆下来之后，打开机器人，启动上位机，点击舵机回中。此时，3 号舵机的中位就

调好了。（注意要拿好刚拆下来的部分，防止开机时机械臂恢复初始动作误伤）



7) 舵机中位好之后，需再将机械臂断电，再将对应的主舵盘和副舵盘以“十”字的形态安装上去。



注意：此时舵机已中位，切勿用力掰动舵机主轴，若舵盘“十”字没有安装到位，需重复本文[大偏差调节](#)；

- 8) 安装好之后，将 U 型支架按两边舵盘的孔位对齐，拧上螺丝。



- 9) 接下来，请将设备开机，重复本文“[小偏差调节](#)”的步骤，对偏差进行微调即可。

5. 舵机调试工具的使用

在我们更换新的舵机时，就需要使用舵机调试工具设置舵机的 ID、中位等参数，关于舵机的 ID 编号，可查看上文“[舵机控制节点介绍](#)”进行了解。

在设置进行舵机调试之前，需要注意，**舵机接口只能单独连接一个舵机**，若串联多个舵机，则会导致所有舵机参数都相同，从而无法控制各个舵机单独转动。

5.1 舵机调试界面简介

1) 参照教程“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”将机器人与远程桌面工具建立连接。

2) 打开终端输入指令。

```
cd ~/software/servo_tool/servo_tool
```

```
cd ~/software/servo_tool/servo_tool
```

3) 然后输入指令，点击“OK”，打开后需确认**总线舵机接口单独连接了一个舵机**。

```
python3 main.py
```

```
python3 main.py
```



4) 本界面分为两个区域，①为舵机参数设置区域，②为舵机设置生效按钮；其参数说

明如下图及表所示：



按钮图标	功能说明
	点击，即可读取舵机当前参数。
	点击，即可设置当前舵机的参数。
	点击，即可使舵机恢复默认参数。
	点击，即可退出舵机调试工具。
舵机ID 1	可对舵机 ID 进行设置，设置范围为（-1~255）
舵机偏差 0	可对舵机偏差进行调整。
温度范围 	可对舵机工作温度范围调整，一般保持默认 85°C 即可。

 角度范围 0 1000	可对舵机转动位置范围调整, 用户可根据实际需求设置舵机转动范围, 在 ArmPi Ultra 机械臂中舵机的转动范围为 0~1000。
 电压范围 4.5V 14V	可对舵机的工作电压范围进行调整, 用户可限制舵机的工作电压, 给舵机提供电压保护。
当前舵机ID: 当前角度: 当前电压: 当前温度:	显示舵机当前 ID、位置、电压、温度。
 舵机调试 500 中位	拖动滑杆, 即可调整舵机的转动位置; 按下中位按键即可回到 500 的位置。

5.2 舵机调试操作步骤

- 1) 将舵机单独连接至舵机接口中。
- 2) 打开终端输入指令, 打开舵机调试工具, 打开后需确认**总线舵机接口单独连接了一个舵机。**

```
python3 ~/software/servo_tool/servo_tool/main.py
```

```
python3 ~/software/servo_tool/servo_tool/main.py
```



3) 点击“读取”按钮。



4) 此处以 ID3 号舵机为例，设置舵机 ID，其它参数可保持默认，用户也可根据需求自行设置。



5) 设置完成后，点击设置即可。



6) 舵机 ID 设置完成后，可前往“[大偏差调节](#)”教程拆卸与组装舵机。